

AquaMind AI

Software Design Document (SDD)

PHASE 1 — Standalone Laptop Telemetry Digital Twin

Version 2.0 | Document Status: Draft for Engineering Review | July 2026

*This document is the Phase 1 extract of the full AquaMind AI SDD, covering only the standalone laptop digital twin.
See the companion Phase 2 document for the distributed / cloud-scale design.*

1. Executive Summary

AquaMind AI is a digital-twin platform for AI data-centre operations that continuously ingests compute telemetry, simulates rack- and facility-level thermal/water behaviour, retrieves relevant operational memory, and produces AI-generated recommendations for cooling and capacity decisions. The system combines a FastAPI orchestration layer, a CockroachDB operational and vector memory store, Amazon Bedrock for reasoning and embeddings, and a React dashboard for visualisation.

Phase 1 delivers this workflow end-to-end using only the developer's own laptop:

- Telemetry is collected from the developer's own laptop (CPU, GPU, RAM, disk, battery, fan, thermal sensors) instead of real data-centre hardware or OpenDC.
- The digital twin, water thermodynamic model, memory engine, and AI decision agent all run against this real (but small-scale) telemetry stream.
- Phase 1 is fully self-contained: it can run entirely offline/local (SQLite fallback, mockable Bedrock calls, single-node CockroachDB free tier) and does not require OpenDC, AWS production infrastructure, or multi-node deployment to demonstrate the complete workflow end-to-end.
- Every module built in Phase 1 (Telemetry Collector, Digital Twin Engine, Water Model, Memory Engine, AI Decision Agent, Dashboard, Reporting) is a working, shippable product on its own — not a stub or prototype to be thrown away.

Phase 1 is architected as a subset of the full two-phase design (same data contracts, same API shapes, same database schema) so that nothing built here needs to be rewritten to reach Phase 2.

1.1 Goals

- Deliver a fully working, demoable product at the end of Phase 1 using only a laptop and free-tier cloud services.
- Prove the water/thermal reasoning and AI recommendation loop on real (if small-scale) telemetry before investing in data-centre simulation infrastructure.
- Design every component so that scaling to Phase 2 is a matter of configuration, deployment topology, and additional data sources — not a rewrite.
- Provide an auditable, explainable AI reasoning trail (memory + recommendation provenance) suitable for operational trust.

1.2 Non-Goals (out of scope for this document)

- Procurement or physical build-out of real data-centre hardware.
- Detailed CFD (computational fluid dynamics) modelling — covered only at a conceptual level under Future Scope.
- Billing/procurement integration with specific utility providers.
- Distributed/cloud-scale deployment, OpenDC/CloudSim integration, and multi-agent orchestration — see the companion Phase 2 document.

2. Requirements Specification

Each functional requirement has an ID for traceability into test cases.

2.1 Functional Requirements — Phase 1 (Standalone Laptop)

ID	Requirement	Priority
FR-1.1	The system shall collect CPU, GPU, RAM, disk, battery, and fan telemetry from the local laptop at a configurable polling interval (default 5s).	Must

ID	Requirement	Priority
FR-1.2	The system shall normalise raw OS-level metrics into a common JSON telemetry schema shared with Phase 2.	Must
FR-1.3	The system shall buffer telemetry locally (SQLite) if the API or database is temporarily unreachable, and replay on reconnect.	Must
FR-1.4	The Digital Twin Engine shall estimate rack-equivalent utilisation and thermal load from a single-device telemetry stream using a configurable synthetic rack profile.	Must
FR-1.5	The Water Model shall compute an estimated water usage effectiveness (WUE) and cooling demand from the Digital Twin's thermal output plus (live or cached) weather data.	Must
FR-1.6	The Memory Engine shall summarise and embed each significant event and store it in CockroachDB with a retrievable vector index.	Must
FR-1.7	The AI Decision Agent shall retrieve top-K similar past memories and produce a natural-language recommendation with a confidence score.	Must
FR-1.8	The Dashboard shall visualise live telemetry, water/cooling estimates, and the latest AI recommendation.	Must
FR-1.9	The system shall expose a CLI mode that reproduces the full workflow without a browser, for headless operation.	Should
FR-1.10	The Reporting module shall export a daily summary (CSV/PDF) of telemetry, water estimates, and recommendations.	Should
FR-1.11	The system shall function with Bedrock calls mocked/disabled (local rules-based fallback) so Phase 1 has zero mandatory paid cloud dependency.	Should

2.2 Non-Functional Requirements — Phase 1 Targets

Category	Phase 1 Target
Availability	Best-effort, single process; local buffering covers transient failures
Latency (ingest → recommendation)	< 5s on local hardware
Scalability	Single device / single process
Data Retention	Local disk, unbounded until manually pruned
Security	Local-only, no external exposure by default
Observability	Structured logs to local file
Cost	\$0 mandatory spend (free tiers, mockable Bedrock)
Portability	Runs on macOS/Windows/Linux laptop

3. Detailed System Architecture

The architecture shares the same conceptual workflow used across both phases — collect telemetry, simulate, estimate cooling/water, retrieve similar memories, generate an AI recommendation, store the new memory, and display it on the dashboard.

3.1 Phase 1 Architecture — Standalone Laptop Digital Twin

Phase 1 intentionally runs as a single deployable unit. The FastAPI application, Digital Twin Engine, Water Model, Memory Engine, and AI Decision Agent all execute in one process (or one container) on the developer's laptop or a single low-cost VM. CockroachDB runs as a single-node free-tier serverless cluster. The AI Decision Agent is a single Bedrock call (or a local rules-based fallback when Bedrock is unavailable/disabled), rather than the multi-agent system used in Phase 2.

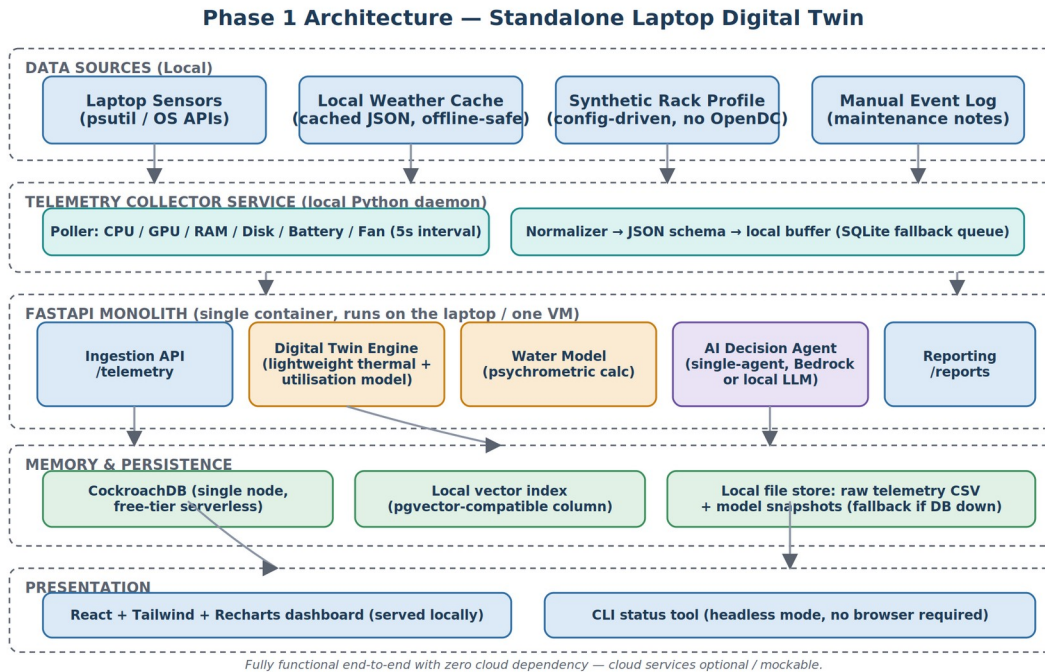


Figure 1 — Phase 1 Architecture: fully standalone, laptop-only digital twin.

3.1.1 Why Phase 1 works completely on its own

- No dependency on OpenDC/CloudSim — a config-driven synthetic rack profile substitutes for simulated data-centre racks.
- No dependency on live weather API — a cached/offline weather snapshot is used, refreshed opportunistically.
- No dependency on multi-node CockroachDB or AWS — single-node serverless free tier is sufficient.
- No mandatory dependency on Amazon Bedrock — a deterministic rules-based recommendation fallback keeps the loop closed if Bedrock credentials are not configured.
- The dashboard and CLI both run locally, so the entire demo ("run a workload, watch utilisation rise, see a water/cooling estimate, get an AI recommendation, see it remembered next time") is achievable with nothing more than a laptop and an internet connection.

4. CockroachDB Memory Architecture

CockroachDB serves two roles simultaneously: an ACID-compliant operational database for structured telemetry/event data, and a vector store for semantic memory retrieval used by the AI Decision Agent (Retrieval-Augmented Generation).

4.1 Write Path

Every significant telemetry reading or event flows through the Memory Engine, which produces a short natural-language summary, generates a Bedrock embedding for that summary, and inserts both the structured row and the embedding vector into CockroachDB in a single transaction.

4.2 Schema Groups

- Operational tables — telemetry, racks, weather, water_model: high write volume, short-lived hot data.

- Event tables — incidents, maintenance, recommendations, audit_log: lower volume, longer retention, human/AI-facing.
- Conversational memory — conversations, embeddings: the substrate for RAG retrieval.

4.3 Read / Retrieval Path (RAG)

When the AI Decision Agent needs context, it embeds its query, issues a cosine-similarity vector search against the embeddings table (top-K, typically K=5), and receives ranked memories that are inserted into the reasoning context window alongside the current telemetry state.

4.4 Consistency and Resilience — Phase 1

Phase 1 runs a single node with default `SERIALIZABLE` isolation; no multi-region configuration is needed. Memory tiering (hot/warm/cold) is a Phase 2 concern — data volume from a single laptop is small enough to retain indefinitely on a single node.

5. AI Decision Agent (Phase 1 — Single Agent)

Phase 1 uses a single AI Decision Agent (one Bedrock call, or a deterministic rules-based fallback). This single-agent implementation reasons over the Water Model's thermodynamic output and retrieved memories to produce a recommendation. All calls go through a shared, versioned tool layer (function-calling tools) rather than accessing data stores directly: `get_telemetry()`, `run_simulation()`, `query_memory()`, `compute_water_model()`, `write_memory()`. This keeps agent logic declarative and testable, and ensures the same tool implementations can be reused unchanged when Phase 2 promotes this logic to a peer agent inside a multi-agent system.

6. Sequence Diagram

The sequence diagram below shows the end-to-end flow from a telemetry reading to a stored, displayed AI recommendation. In Phase 1, the 'AI Decision Agent' box represents a single Bedrock call (or local fallback) and the 'Digital Twin Engine' / 'Water Model' boxes run the laptop-scale models.

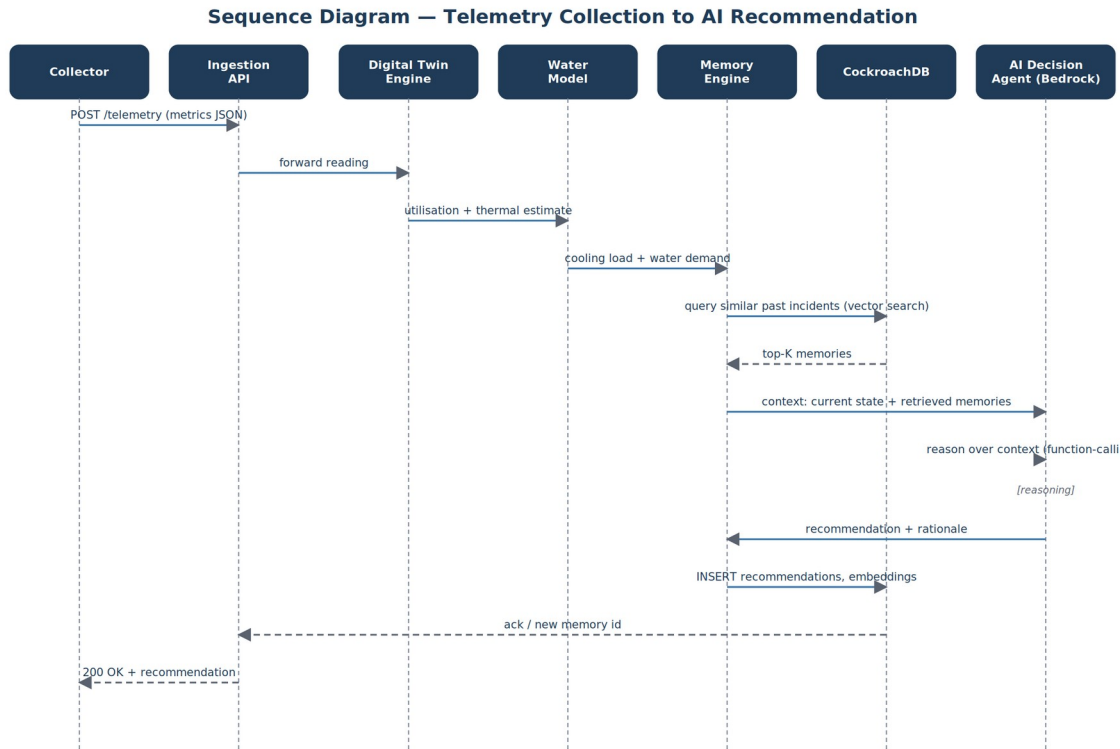


Figure 6 — Sequence: Telemetry Collection → Digital Twin → Water Model → Memory Retrieval → AI Recommendation → Persistence → Dashboard.

6.1 Key Interaction Notes

- Steps 1–3 (Collector → Ingestion → Digital Twin → Water Model) are synchronous and complete in well under a second.
- Step 4 (vector similarity search) is the main latency contributor as memory volume grows.
- The reasoning loop (function-calling) may issue multiple internal tool calls before returning; this is shown as a single self-referential step for clarity.
- Persistence of the new memory (INSERT into recommendations, embeddings) happens before the response is returned to the original collector, guaranteeing the recommendation is retrievable by the next request.

7. Class Diagram

The class diagram below shows the core domain model. Phase 1 instantiates a single concrete Agent (the Water & Cooling logic).

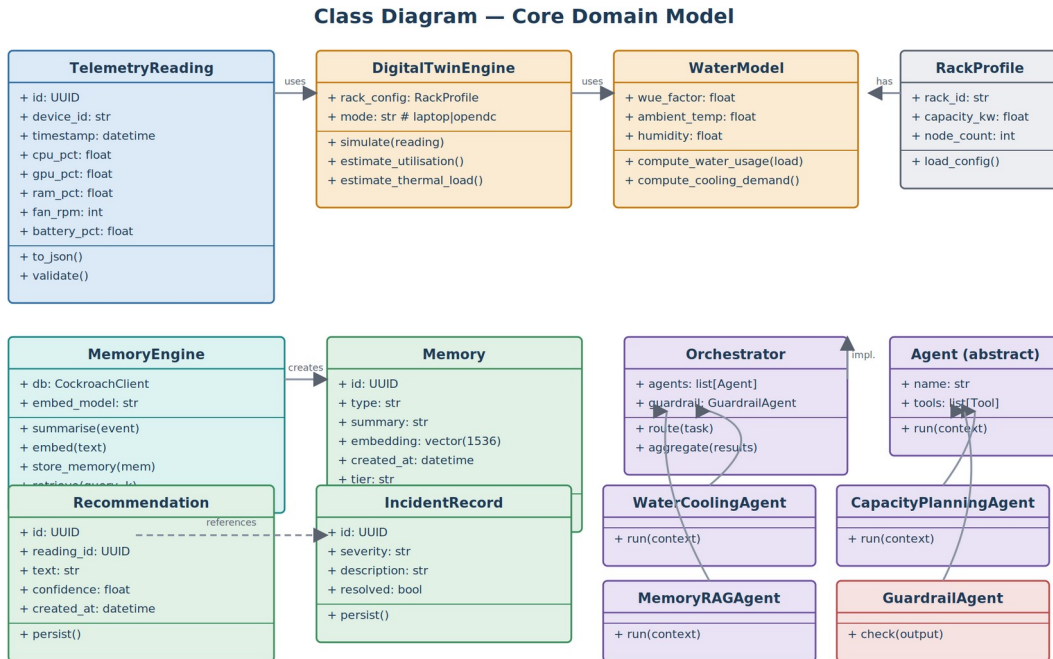


Figure 7 — Core domain class diagram.

7.1 Notable Design Decisions

- DigitalTwinEngine.mode is a strategy flag ('laptop' vs 'opencdc') rather than a separate class hierarchy at the top level.
- Agent is an abstract base class from day one, even though Phase 1 only ever instantiates one concrete subclass — this avoids a Phase 2 rewrite of the reasoning layer.
- Memory is a first-class object with an explicit tier attribute, so tiering logic introduced later requires no schema migration, only a background job.

8. Component Diagram

The component diagram groups the system into Presentation, Application, and Data/Integration tiers. In Phase 1, all Application-tier components run inside one process.

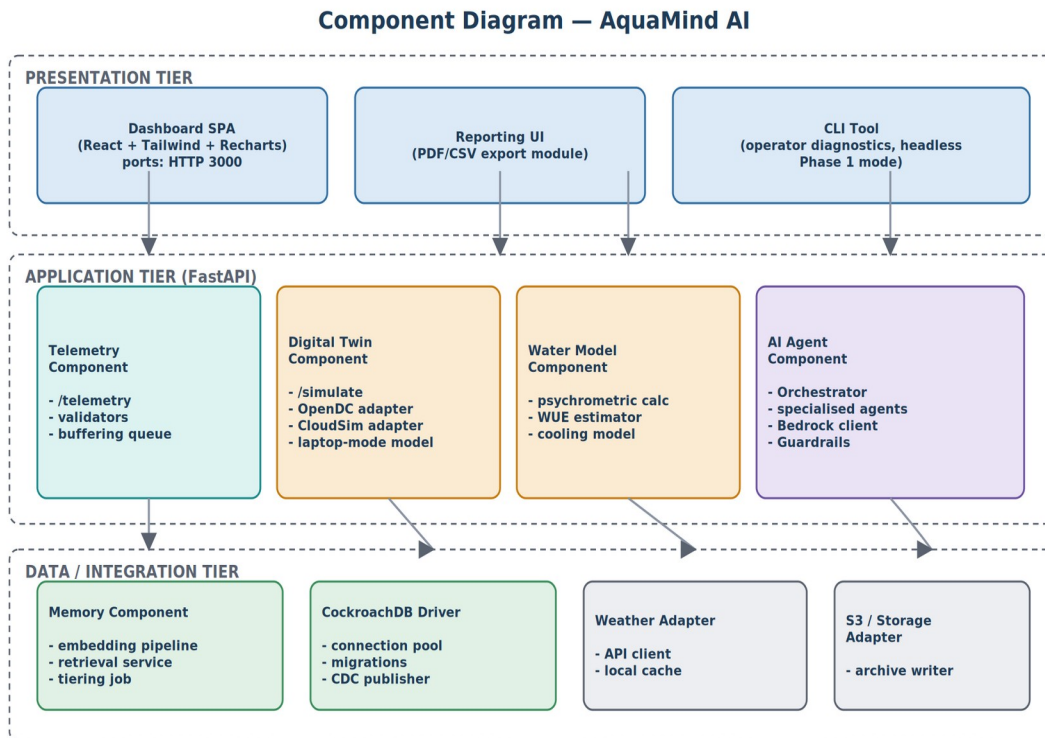


Figure 8 — Component diagram with presentation, application, and data/integration tiers.

9. ER Diagram (Database Schema)

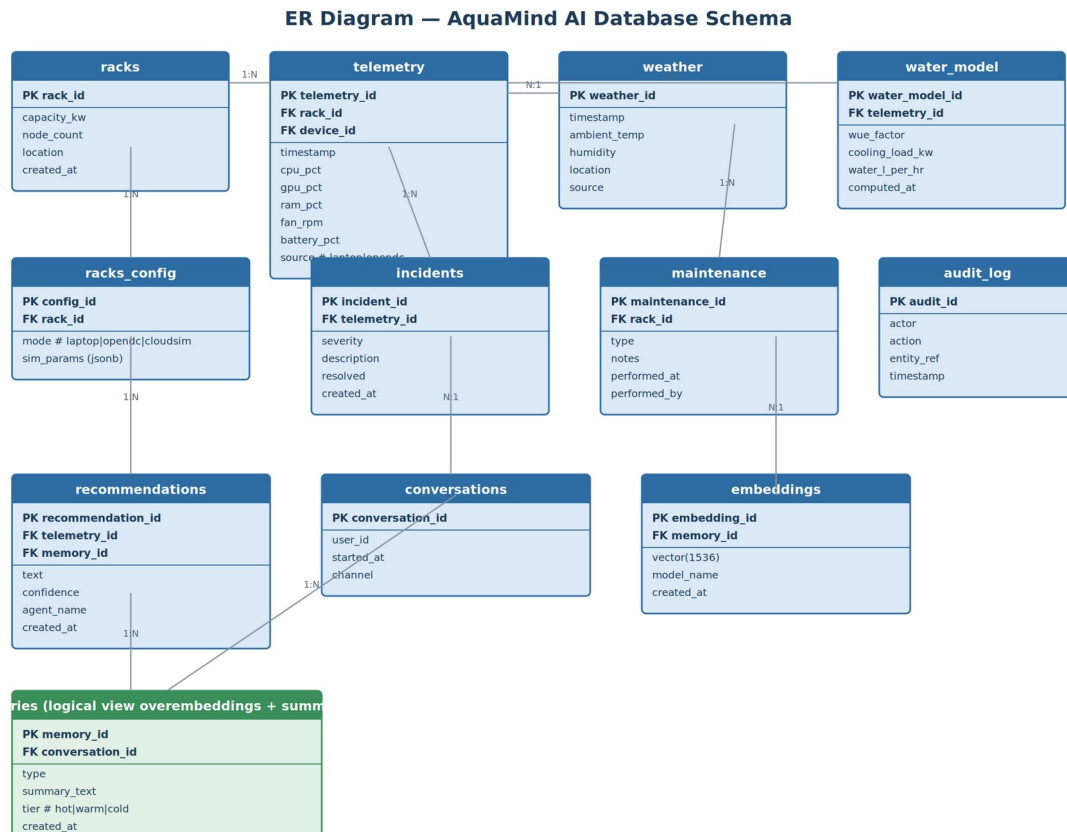


Figure 9 — Entity-relationship diagram for the AquaMind AI database.

9.1 Table Definitions

Table	Key Columns	Phase 1 Notes
racks	rack_id (PK), capacity_kw, node_count, location, created_at	One synthetic rack representing the laptop.
telemetry	telemetry_id (PK), rack_id (FK), device_id (FK), timestamp, cpu_pct, gpu_pct, ram_pct, fan_rpm, battery_pct, source	source = 'laptop'.
weather	weather_id (PK), timestamp, ambient_temp, humidity, location, source	Uses a cached snapshot.
water_model	water_model_id (PK), telemetry_id (FK), wue_factor, cooling_load_kw, water_l_per_hr, computed_at	Output of the Water Thermodynamic Model (Section 12).
racks_config	config_id (PK), rack_id (FK), mode, sim_params (jsonb)	mode = 'laptop'.
incidents	incident_id (PK), telemetry_id (FK), severity, description, resolved, created_at	Flagged by threshold rules in Phase 1.
maintenance	maintenance_id (PK), rack_id (FK), type, notes, performed_at, performed_by	Manual event log.
audit_log	audit_id (PK), actor, action, entity_ref, timestamp	Append-only; required for security design (Section 17).
recommendations	recommendation_id (PK), telemetry_id (FK), memory_id (FK), text, confidence, agent_name, created_at	agent_name = 'rules_fallback' or 'bedrock_single'.

Table	Key Columns	Phase 1 Notes
	created_at	
conversations	conversation_id (PK), user_id, started_at, channel	Groups related memory entries.
embeddings	embedding_id (PK), memory_id (FK), vector(1536), model_name, created_at	Vector column indexed with CockroachDB's vector index (HNSW).
memories	memory_id (PK), conversation_id (FK), type, summary_text, tier, created_at	Logical view combining embeddings + summaries; tier field unused until Phase 2.

This schema is designed to be used unchanged into Phase 2 — no columns need to be added, removed, or renamed, preserving forward compatibility for the dashboard and API.

10. API Specification (FastAPI / OpenAPI)

All endpoints are versioned under /api/v1 and use the shared telemetry schema so no breaking changes are needed when extending to Phase 2.

10.1 Core Endpoints (Phase 1 — available from day one)

Method & Path	Description	Request Body	Response
POST /api/v1/telemetry	Ingest one telemetry reading	TelemetryReading JSON	202 Accepted, {telemetry_id}
GET /api/v1/telemetry/latest	Latest reading for the local device	—	TelemetryReading JSON
POST /api/v1/simulate	Run the Digital Twin against the latest (or supplied) reading	{telemetry_id?}	{utilisation, thermal_load_kw}
GET /api/v1/water-model/latest	Latest water/cooling estimate	—	WaterModel JSON
POST /api/v1/recommend	Trigger the AI Decision Agent for a recommendation	{telemetry_id}	Recommendation JSON
GET /api/v1/memory/search	Vector search over stored memories	?q=<text>&k=5	list[Memory]
GET /api/v1/dashboard/summary	Aggregated data for the dashboard	—	DashboardSummary JSON
GET /api/v1/reports/daily	Generate/download the daily report	?format=csv pdf	file stream

10.2 Shared TelemetryReading Schema (OpenAPI-style)

TelemetryReading { telemetry_id: string (uuid, server-generated); device_id: string; rack_id: string; timestamp: string (ISO-8601); cpu_pct: number (0-100); gpu_pct: number (0-100, nullable); ram_pct: number (0-100); fan_rpm: integer (nullable); battery_pct: number (0-100, nullable); source: enum [laptop, opendc, cloudsim, iot] }

10.3 Error Model

All endpoints return RFC 7807 problem+json on error: {type, title, status, detail, instance}. Validation errors (422) include a field-level errors array.

10.4 Authentication — Phase 1

No authentication is required by default (local-only binding to 127.0.0.1); an optional local API token can be enabled.

11. Memory Lifecycle Design

Memory in AquaMind AI is not just conversation history — it is the operational record that lets the AI Decision Agent learn from past incidents. Its lifecycle has four stages.

11.1 Stage 1 — Capture

A candidate memory is created whenever: (a) a recommendation is generated, (b) an incident is flagged, (c) a maintenance event is logged, or (d) a user provides feedback on a recommendation.

11.2 Stage 2 — Summarise & Embed

The Memory Engine converts the structured event into a short natural-language summary (template-based in Phase 1 for determinism and zero cost), then generates a 1536-dimension embedding via Bedrock Titan Embeddings (or a local sentence-transformer fallback when Bedrock is disabled).

11.3 Stage 3 — Store & Index

The summary, embedding, and structured payload are inserted into CockroachDB in a single transaction. The vector column is covered by a vector index (HNSW) to keep similarity search sub-100ms even as the table grows.

11.4 Stage 4 — Retrieve

Retrieve: top-K cosine similarity search, filtered by recency and optionally by rack/site. (Ageing and retirement of memories into warm/cold tiers is a Phase 2 concern — see the companion Phase 2 document.)

11.5 Provenance and Explainability

Every recommendation stores the IDs of the memories that were retrieved to produce it, so any recommendation can be explained after the fact by re-fetching its cited memories — a requirement for operational trust (Section 1.1, Goal 4).

12. Digital Twin Design

The Digital Twin Engine is the abstraction that lets the same downstream pipeline (Water Model → Memory → AI Agent) operate identically whether the underlying data source is a single laptop or (in Phase 2) a simulated data centre.

12.1 Interface

```
class DigitalTwinEngine: def simulate(self, reading: TelemetryReading) -> TwinState: ... def
estimate_utilisation(self) -> float: ... def estimate_thermal_load(self) -> float: # kW
```

12.2 Phase 1 — Laptop Mode

A configurable RackProfile (capacity_kw, node_count) scales the single device's utilisation into an equivalent rack-level figure. For example, a laptop at 80% CPU utilisation mapped onto a profile with capacity_kw=5 and node_count=1 produces a directly proportional thermal load estimate; the profile can be tuned to represent a larger notional rack for more meaningful water-model output.

12.3 TwinState Output Contract (shared with Phase 2)

```
TwinState { rack_id: string; utilisation_pct: number; thermal_load_kw: number; power_draw_kw: number; mode:
enum [laptop, opendc, cloudsim] }
```

13. Water Thermodynamic Model

The Water Model converts a Digital Twin's thermal load into an estimate of cooling energy demand and water consumption, using standard data-centre efficiency metrics.

13.1 Key Metrics and Equations

Cooling Load (Q): the heat that must be removed, derived directly from the Digital Twin's `thermal_load_kw` output (assumed equal to IT power draw under steady state).

$$Q_{cooling} (kW) = thermal_load_kw \times (1 + PUE_thermal_overhead)$$

Power Usage Effectiveness (PUE): ratio of total facility energy to IT equipment energy. Default assumption in Phase 1: 1.4 (typical air-cooled small facility).

$$PUE = Total_Facility_Energy / IT_Equipment_Energy$$

Water Usage Effectiveness (WUE): litres of water consumed per kWh of IT energy, the model's key output metric.

$$WUE (L/kWh) = Water_Consumed_L / IT_Equipment_Energy_kWh$$

Estimated Water Consumption Rate: combines WUE, cooling load, and ambient wet-bulb conditions (approximated from temperature and humidity) using a simplified psychrometric evaporation model.

$$Water_L_per_hr = WUE \times IT_Equipment_Energy_kWh \times f(ambient_temp, humidity)$$

where $f(\dots)$ is a monotonic scaling factor: higher ambient temperature and lower humidity increase evaporative cooling water demand, approximated via a lookup table derived from ASHRAE psychrometric charts.

13.2 Assumptions — Phase 1

- A fixed, configurable PUE and a simplified evaporative-cooling assumption (single lookup-table factor) — sufficient to demonstrate the reasoning loop, not for utility-bill-grade accuracy.
- The model's output is treated as a decision-support estimate, not a certified engineering measurement; this caveat is surfaced in the dashboard UI.

13.3 Validation Approach

Model outputs are sanity-checked against published industry WUE benchmarks (typically 0.5–2.0 L/kWh for air-cooled facilities) as an automated test assertion, flagging any configuration that produces physically implausible results.

14. Laptop Telemetry Collection Design

The Telemetry Collector is a lightweight local daemon (Python) that polls OS-level metrics and pushes normalised readings to the Ingestion API. It is the first component built, since every other module depends on it having real data to operate against.

14.1 Metrics Collected

Metric	Source (cross-platform)	Notes
CPU utilisation	<code>psutil.cpu_percent()</code>	Per-core and aggregate
GPU utilisation	<code>nvidia-smi</code> / <code>pynvml</code> (if present), else null	Gracefully degrades on laptops without a discrete GPU
RAM utilisation	<code>psutil.virtual_memory()</code>	—
Disk I/O	<code>psutil.disk_io_counters()</code>	Used as a secondary load signal
Battery percentage	<code>psutil.sensors_battery()</code>	Proxy for power draw on battery-powered

Metric	Source (cross-platform)	Notes
		devices
Fan speed / thermal	OS-specific (WMI on Windows, IOKit on macOS, /sys/class/hwmon on Linux), else estimated from CPU load	Used directly by the Water Model as a thermal signal

14.2 Collector Reliability

- Polling interval is configurable (default 5s).
- If the Ingestion API is unreachable, readings are appended to a local SQLite queue and replayed in order once connectivity returns, preserving timestamp ordering.
- The collector process is designed to run as a background service/daemon with automatic restart (systemd/launchd/Task Scheduler), so Phase 1 can run unattended for extended demo periods.

14.3 Forward Compatibility

The identical collector binary is designed to later be distributed to N edge devices without modification — only the ingestion endpoint and an added device_id/site_id in the payload change, both of which are already present in the shared schema.

15. Vector Memory & RAG Flow

Retrieval-Augmented Generation (RAG) is what allows the AI Decision Agent to reason with institutional memory rather than a stateless single-shot prompt.

15.1 Flow

- Current context is assembled: latest TwinState, latest WaterModel output, and any open incidents.
- A retrieval query is constructed from this context (e.g. "high thermal load, rising fan speed, low ambient humidity").
- The query is embedded via Bedrock Titan Embeddings, or a local fallback.
- CockroachDB's vector index returns the top-K most similar past memories (cosine similarity), optionally filtered to the same rack/site.
- Retrieved memories are ranked by a blend of similarity score and recency, then formatted into the prompt context (Section 16).
- The AI Decision Agent reasons over current context + retrieved memories and produces a recommendation with cited memory IDs.

15.2 Embedding Model — Phase 1

Default: Bedrock Titan Text Embeddings V2 (1536 dimensions) when Bedrock is configured; a local sentence-transformer (all-MiniLM-L6-v2, 384 dimensions, padded/truncated to a consistent stored dimension) as a zero-cost offline fallback.

16. Amazon Bedrock Prompt Architecture

Prompts are structured, versioned, and stored as templates (not hard-coded strings) so that prompt changes are auditable and testable independently of application code.

16.1 Prompt Template Structure


```

├── dashboard.py | ├── reports.py | ├── digital_twin/ | ├── laptop_mode.py |
├── water_model/ | ├── thermo.py | ├── memory_engine/ | ├── summarise.py |
├── embed.py      # Bedrock + local fallback | ├── store.py | ├── agent/ |
bedrock_client.py | ├── rules_fallback.py | ├── dashboard/      # React app (local dev server)
├── shared/        # Used by both phases | ├── schemas/        # Pydantic models:
TelemetryReading, TwinState, etc. | ├── db/          # CockroachDB migrations (schema in Section 9) |
├── prompts/       # Versioned Bedrock prompt templates | ├── tests/ | ├── phase1/ | ├── docs/ |
AquaMind_AI_SDD_Phase1.docx # This document

```

21. Implementation Timeline — Phase 1 (Target: 6 weeks)

Week	Milestone
1	Telemetry Collector daemon (poller, normaliser, local SQLite queue); shared schemas defined
2	FastAPI monolith skeleton; /telemetry ingestion; CockroachDB single-node provisioned and migrated
3	Digital Twin Engine (laptop mode) and Water Model (thermodynamic calculations) implemented and unit-tested
4	Memory Engine (summarise, embed, store) with local fallback embeddings; AI Decision Agent (single Bedrock call + rules fallback)
5	React dashboard (live telemetry, water/cooling view, recommendation view); CLI headless mode; Reporting (CSV/PDF)
6	End-to-end hardening, demo scripting, documentation, and Phase 1 sign-off

22. Risk Analysis — Phase 1

Risk	Likelihood	Impact	Mitigation
Laptop telemetry too limited to produce meaningful water-model output for demos	Medium	Medium	Configurable RackProfile scaling (Section 12.2) lets Phase 1 represent a larger notional rack for realistic output
Bedrock cost or access issues block Phase 1 demos	Low	Medium	Local rules-based fallback and local embedding model keep Phase 1 fully functional with zero Bedrock dependency
Scope creep pulls Phase 1 team into premature AWS/OpenDC work	Medium	Medium	Explicit non-goals (Section 1.2) and phase gate (Phase 2 starts only after Phase 1 sign-off)

Appendix A — Glossary

Term	Definition
Digital Twin	A software model that mirrors the behaviour of a physical (or simulated) system in near real time
PUE	Power Usage Effectiveness — ratio of total facility energy to IT equipment energy
WUE	Water Usage Effectiveness — litres of water consumed per kWh of IT energy
RAG	Retrieval-Augmented Generation — grounding an LLM's response in retrieved documents/memories
HNSW	Hierarchical Navigable Small World — a vector index structure for approximate nearest-neighbour search